

LAPORAN UAS MOBILE PROGRAMMING



Nama : ALIF FAKHRUL HAKIM

NIM : 240605110086

Kelas : MOBILE PROGRAMMING (B)

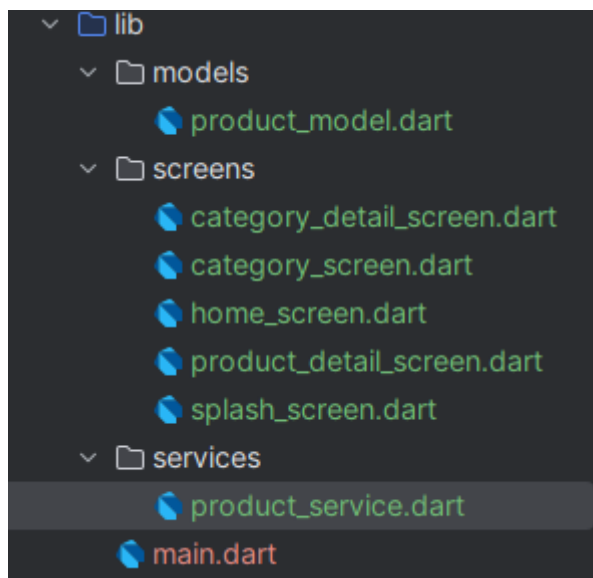
**JURUSAN TEKNIK INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS ISLAM NEGERI MAULANA MALIK IBRAHIM MALANG
GANJIL 2025/2026**

InterRoom: PinjamRuang Interior Support

1. Arsitektur Aplikasi

Aplikasi *Katalog Interior Room* dibangun menggunakan framework Flutter dengan pendekatan arsitektur yang terstruktur dan modular. Struktur arsitektur ini dipilih agar aplikasi mudah dikembangkan, mudah dipelihara, serta mampu memisahkan logika bisnis dengan tampilan antarmuka pengguna. Arsitektur aplikasi ini mengadopsi pola *layered architecture* sederhana dengan memisahkan tiap bagian berdasarkan fungsinya, yaitu: Model, Service/Data Layer, UI Layer, dan Routing/Navigasi.

Berikut adalah Struktur Projectnya:



1.1. Model Layer

Model layer digunakan untuk memetakan data dari API ke dalam bentuk objek yang dapat digunakan Flutter.

Aplikasi menggunakan satu model utama:

Field dalam model:

- id = integer
- title = nama produk
- description = deskripsi lengkap
- image = URL gambar produk (langsung dari API pada key "image")
- price = harga (double)
- category = kategori produk

Model diambil dari JSON menggunakan factory:

factory ProductModel.fromJson(Map<String, dynamic> json)

Model ini memastikan:

- Aplikasi tetap stabil meskipun API berubah format
- Parsing data lebih aman dan terstruktur
- UI dapat mengakses data dengan mudah

1.2. Service Layer

Service layer adalah bagian yang bertanggung jawab dalam melakukan *networking*, yaitu melakukan komunikasi dengan API FakeStoreAPI.

Aplikasi ini mengimplementasikan 3 metode:

- Get All Product
- Get All Category
- Get Product by Category

1.3. UI Layer

UI Layer adalah bagian yang bertanggung jawab menampilkan informasi kepada pengguna. Semua komponen UI berada dalam folder /screens/. Setiap halaman (screen) memiliki fungsi yang berbeda-beda sesuai alurnya.

Terdapat 5 screen dalam aplikasi ini, yaitu:

- **Splash Screen**, halaman pertama yang ditampilkan ketika aplikasi baru dibuka
- **Home Screen**, halaman utama aplikasi yang menampilkan seluruh produk dari API FakeStoreAPI
- **Product Detail Screen**, halaman yang menampilkan informasi lebih lengkap dari produk yang dipilih
- **Category Screen**, halaman untuk menampilkan seluruh kategori dari API FakeStoreAPI
- **Category Detail Screen**, halaman untuk menampilkan produk dari kategori yang dipilih

1.4. State Management

Aplikasi ini menggunakan setState() sebagai state management utama.

Alasan memilih setState:

- Aplikasi tidak memiliki data yang saling terhubung secara global.

- SetState sangat mudah digunakan.
- Cocok untuk aplikasi dengan layar yang tidak terlalu kompleks.
- Perubahan UI bisa langsung di-refresh ketika state berubah.

2. Penjelasan API yang Digunakan

Aplikasi ini menggunakan FakeStoreAPI, yaitu API publik yang sering dipakai untuk latihan dan simulasi aplikasi e-commerce. API ini tidak memerlukan API key sehingga proses integrasinya sangat sederhana. Semua data yang diberikan berupa JSON, sehingga mudah diolah menggunakan model ProductModel di aplikasi. FakeStoreAPI menyediakan data lengkap seperti judul produk, deskripsi, gambar, kategori, dan harga, sehingga cocok untuk aplikasi katalog seperti ini.

Dalam implementasinya, API ini membantu aplikasi menampilkan daftar produk, daftar kategori, serta produk berdasarkan kategori tertentu. Proses pengambilan data dilakukan melalui HTTP GET, lalu data yang diterima dikonversi menjadi objek Dart agar mudah dipakai di tampilan. Dengan cara ini, logika backend tetap sederhana namun UI tetap responsif dan dinamis mengikuti data yang berasal dari server.

Aplikasi ini menggunakan tiga endpoint utama untuk memenuhi kebutuhan fitur: mengambil semua produk, mengambil daftar kategori, dan mengambil produk berdasarkan kategori yang dipilih. Ketiga endpoint tersebut dipanggil melalui kelas ProductService, sehingga pemisahan antara logika data dan UI tetap terjaga.

Endpoint yang digunakan dalam aplikasi adalah sebagai berikut:

- Get /product (digunakan untuk mengambil semua data produk)
- Get /product/categories (digunakan untuk mengambil data kategori produk)
- Get /products/category/{category} (untuk mengambil data produk berdasarkan kategori yang dipilih)

3. Screenshot Hasil Pengujian

3.1. Pengujian Berhasil

- **Splash Screen**

Screenshot:



Penjelasan:

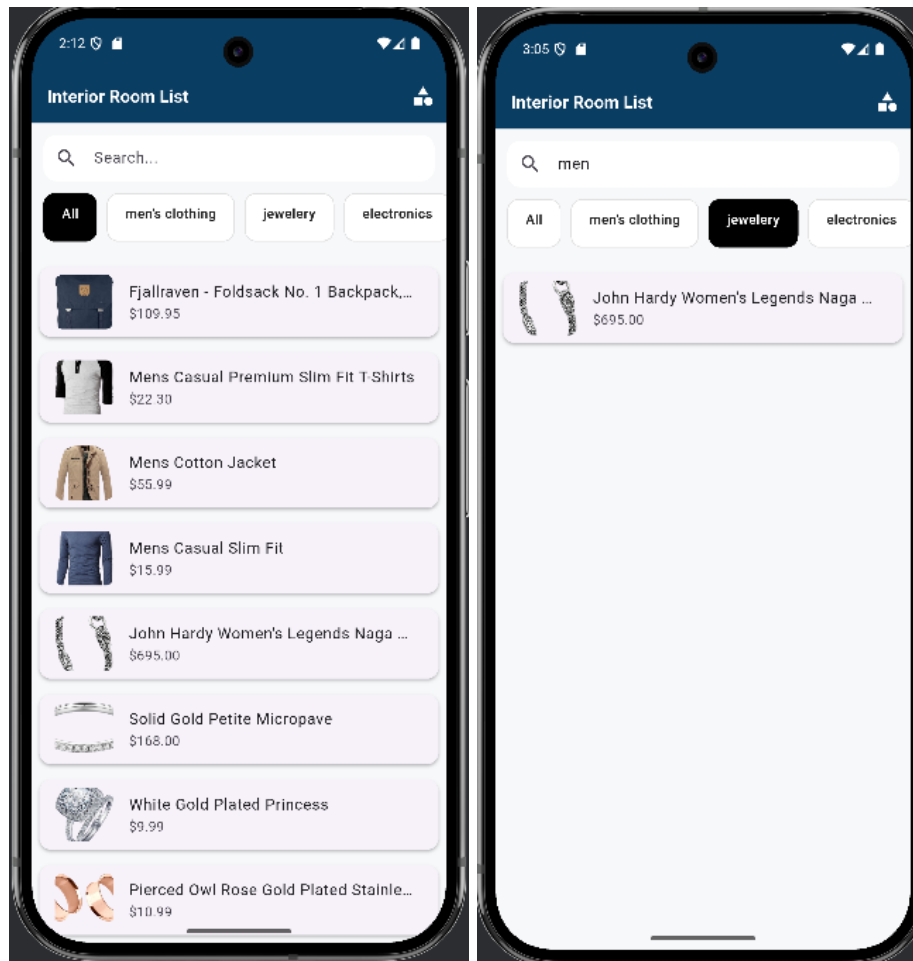
Splash Screen adalah halaman pembuka yang muncul sesaat ketika aplikasi dijalankan, berfungsi menampilkan logo atau identitas aplikasi sekaligus melakukan proses inisialisasi awal.

Cara Kerjanya:

Saat aplikasi mulai berjalan, sistem memanggil halaman Splash terlebih dahulu, kemudian menjalankan proses loading ringan seperti pengecekan koneksi atau persiapan konfigurasi, biasanya menggunakan `Future.delayed` untuk memberikan durasi tampil tertentu. Setelah proses selesai atau waktu tunggu berakhir, aplikasi secara otomatis melakukan navigasi ke Home Screen tanpa membutuhkan interaksi pengguna. Dengan mekanisme ini, splash screen tidak hanya memberikan kesan profesional, tetapi juga membantu menyamarkan waktu pemuatan sehingga transisi ke halaman utama terasa lebih halus.

- **Home Screen**

Screenshot 1:



Penjelasan:

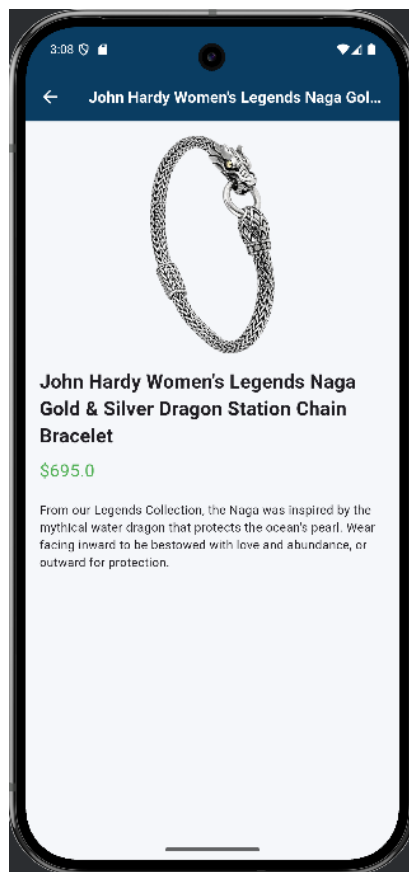
HomeScreen adalah halaman utama aplikasi yang berfungsi sebagai pusat informasi sekaligus titik awal navigasi pengguna. Halaman ini menampilkan daftar produk yang diambil dari API publik menggunakan tampilan list yang bersih dan mudah dibaca. Di bagian atas, terdapat search bar untuk mencari produk berdasarkan kata kunci, serta deretan kategori berupa chip button yang memungkinkan pengguna memfilter data sesuai kategori tertentu. Antarmukanya dirancang agar sederhana dan intuitif sehingga pengguna dapat langsung melihat produk, gambar, harga, serta melakukan eksplorasi tanpa hambatan. Secara keseluruhan, HomeScreen memadukan tampilan visual dengan interaksi fungsional yang membuat pengalaman penggunaan aplikasi menjadi lebih nyaman dan efisien.

Cara Kerjanya:

Saat HomeScreen dibuka, aplikasi memanggil fungsi `fetchProducts()` dari `ProductService` untuk mengambil seluruh data produk dari API. Data yang diterima kemudian disimpan ke dalam state dan ditampilkan dalam bentuk list. Jika

pengguna mengetik pada search bar, sistem melakukan *filter lokal* dengan mencocokkan teks yang diketik dengan judul produk tanpa perlu memanggil API lagi. Ketika pengguna memilih kategori tertentu, aplikasi memanggil fungsi `fetchCategoryProducts(category)` untuk mengambil data produk berdasarkan kategori yang dipilih, kemudian memperbarui tampilan menggunakan `setState`. Semua perubahan, baik melalui pencarian maupun pemilihan kategori, langsung merender ulang UI sehingga pengguna dapat melihat hasil secara real-time. Dengan mekanisme ini, HomeScreen mampu menampilkan data secara dinamis, responsif, dan akurat sesuai input pengguna.

- **Product Detail Screen**



Penjelasan:

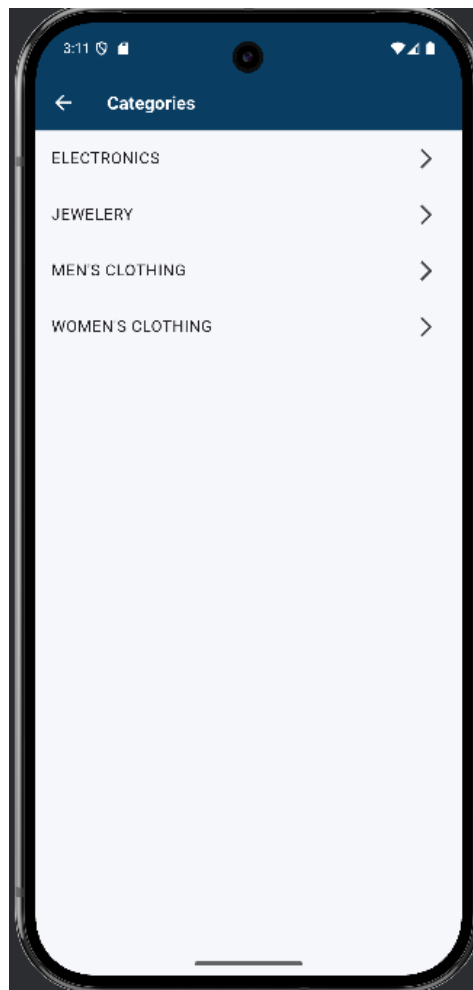
Product Detail Screen adalah halaman yang menampilkan informasi lengkap mengenai satu produk yang dipilih pengguna dari HomeScreen. Tampilan halaman ini dibuat sederhana namun informatif, dimulai dari bagian atas yang menampilkan gambar produk dengan ukuran besar sehingga pengguna dapat melihat detail visual secara jelas. Di bawah gambar, terdapat nama produk dengan ukuran teks lebih

besar agar mudah dibaca, diikuti dengan harga produk yang ditampilkan menggunakan warna hijau untuk memberikan kesan penekanan dan keterbacaan yang baik. Deskripsi produk ditampilkan dalam bentuk paragraf sehingga pengguna dapat memahami fungsi, fitur, dan karakteristik produk secara lebih mendalam. Secara keseluruhan, halaman ini berfungsi sebagai tempat untuk memberikan informasi lengkap sebelum pengguna melakukan keputusan lebih lanjut, serta menjaga konsistensi desain dengan tampilan aplikasi secara umum.

Cara Kerjanya:

Saat pengguna mengetuk salah satu produk dari daftar di HomeScreen, aplikasi mengirimkan data produk tersebut melalui parameter ketika melakukan navigasi menggunakan `Navigator.push`. `Product Detail Screen` menerima objek `ProductModel` yang berisi informasi lengkap seperti `title`, `image`, `price`, dan `description`. Data ini langsung ditampilkan pada widget tanpa perlu memanggil API ulang, sehingga halaman dapat dimuat dengan cepat dan efisien. Gambar produk diambil dari URL menggunakan `Image.network`, dan jika gagal dimuat, widget menyediakan fallback berupa icon placeholder. Seluruh informasi kemudian dirender menggunakan widget-widget seperti `Text`, `Padding`, dan `Column`. Karena semua data sudah diterima dari halaman sebelumnya, `Product Detail Screen` bekerja sepenuhnya secara lokal, ringan, dan responsif, dengan update UI yang hanya terjadi sekali ketika halaman pertama kali ditampilkan.

- **Category Screen**



Penjelasan:

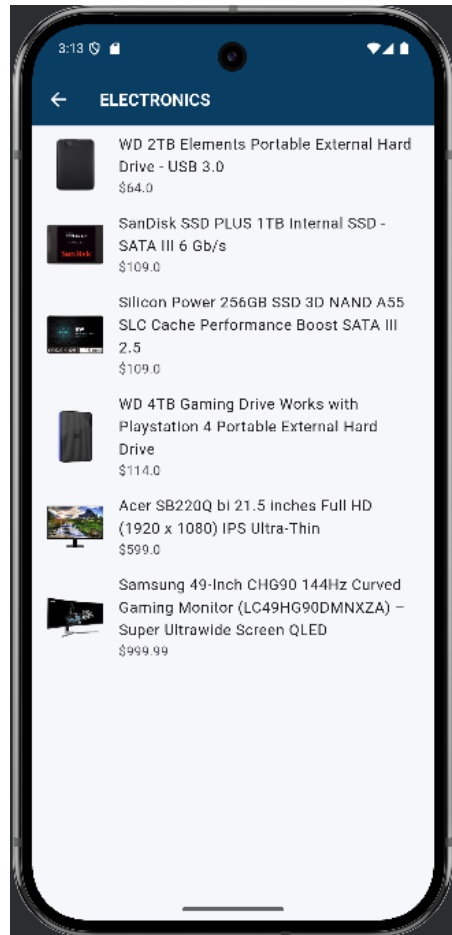
Category Screen adalah halaman yang menampilkan daftar kategori produk yang dapat dipilih pengguna sebelum masuk ke halaman daftar produk. Halaman ini berfungsi sebagai menu awal untuk memfilter produk berdasarkan jenisnya.

Cara Kerjanya:

Category Screen bekerja dengan menampilkan daftar kategori produk dalam bentuk list yang dapat ditekan oleh pengguna. Setiap kategori, seperti *Electronics*, *Jewelery*, *Men's Clothing*, dan *Women's Clothing*, ditampilkan menggunakan komponen list seperti ListTile yang memiliki fungsi onTap. Ketika pengguna memilih salah satu kategori, aplikasi akan menjalankan proses navigasi ke halaman berikutnya, yaitu Product Screen, sambil membawa informasi kategori yang dipilih. Informasi ini kemudian digunakan oleh Product Screen untuk mengambil data produk dari API sesuai kategori tersebut. Setelah data berhasil diambil, halaman Product Screen akan menampilkan daftar produk yang hanya berasal dari kategori yang dipilih pengguna.

sebelumnya. Dengan alur ini, Category Screen berfungsi sebagai pintu awal untuk memfilter produk dan mengarahkan pengguna ke halaman produk yang relevan.

- **Category Detail Screen**



Penjelasan:

Category Detail Screen adalah halaman yang menampilkan daftar produk berdasarkan kategori yang dipilih oleh pengguna pada Category Screen. Tampilan halaman ini biasanya berupa daftar item produk lengkap dengan gambar, nama, dan harga. Tujuannya adalah memberikan pengalaman yang lebih terfokus, sehingga pengguna hanya melihat produk yang relevan dengan kategori yang sedang mereka jelajahi. Desainnya dibuat sederhana dan bersih agar mudah dibaca, serta memberikan navigasi yang jelas untuk kembali ke halaman sebelumnya atau masuk ke halaman detail produk tertentu.

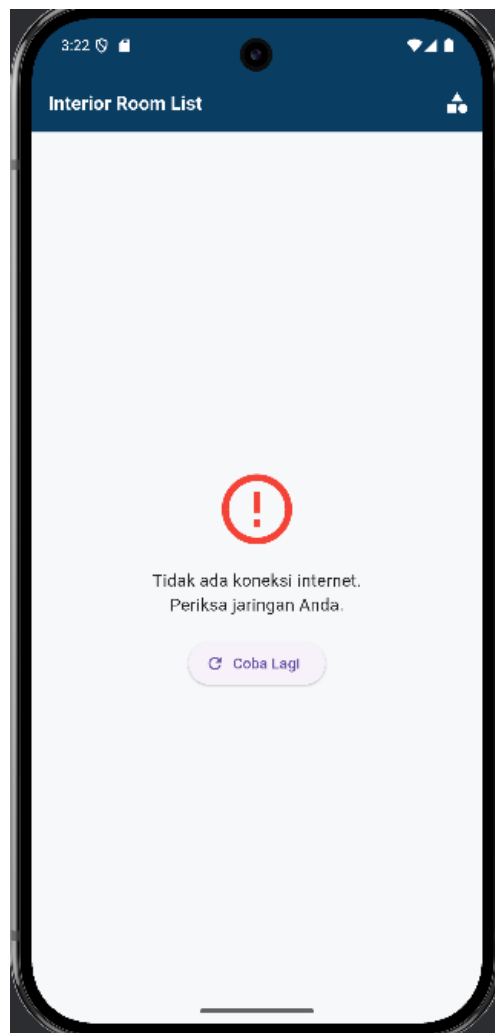
Cara Kerjanya:

Category Detail Screen bekerja dengan menerima data kategori yang dikirim dari Category Screen melalui proses navigasi. Ketika halaman ini dibuka, aplikasi langsung

memanggil API GET `/products/category/{category}` untuk mengambil seluruh produk yang termasuk dalam kategori tersebut. Setelah data diterima, halaman melakukan proses parsing dan menampilkannya dalam bentuk list. Setiap item list memiliki fungsi interaktif: ketika pengguna menekan salah satu produk, aplikasi akan menavigasi ke Product Detail Screen sambil membawa seluruh data produk tersebut. Proses ini membuat alur aplikasi lebih dinamis, karena setiap tampilan langsung menyesuaikan sesuai kategori yang dipilih pengguna, dan data selalu ditampilkan secara real-time berdasarkan hasil API.

3.2. Pengujian Tidak Berhasil (Error Handling)

- Error Network Connection



Penjelasan:

Ketika koneksi internet pengguna lemah dan tidak menyanggupi untuk meng GET API maka akan memunculkan teks “Tidak ada koneksi internet. Periksa

Jaringan Anda” dengan ada tombol Coba lagi untuk mencoba apakah sinyalnya sudah kuat atau belum.

- **Error Data Not Found**



Penjelasan:

Ketika pengguna ingin mencari data tapi data tersebut tidak ada, maka aplikasi akan menampilkan “Tidak ada data ditemukan”

4. Kesimpulan

Aplikasi katalog produk berbasis Flutter ini dibangun menggunakan FakeStoreAPI sebagai sumber data utama untuk menampilkan daftar produk, kategori, serta detail barang. Alur aplikasi dimulai dari *Splash Screen* sebagai tampilan pembuka yang melakukan inisialisasi awal sebelum memasuki *Home Screen* sebagai halaman utama. Di halaman ini, pengguna dapat melihat daftar produk, melakukan pencarian, memfilter berdasarkan kategori, serta

melakukan navigasi ke halaman detail produk maupun halaman kategori. Setiap elemen UI dirancang agar mudah digunakan dan menampilkan informasi secara jelas, termasuk gambar produk, harga, dan judul barang.

Aplikasi ini juga dilengkapi dengan *error handling* yang baik untuk memastikan kenyamanan pengguna saat terjadi gangguan seperti masalah jaringan atau kegagalan memuat data API. Jika error muncul, aplikasi menampilkan pesan yang jelas serta memberi opsi untuk melakukan pemuatan ulang. Selain itu, terdapat *RefreshIndicator* untuk memperbarui data secara manual. Secara keseluruhan, aplikasi ini menawarkan pengalaman yang lengkap dan responsif, mulai dari pengambilan data API, navigasi antar-halaman, pencarian, filter kategori, hingga tampilan detail produk, menjadikannya contoh yang baik untuk proyek Flutter berbasis API publik.